VLSI Formal Verification MCP Server

Technical Design Document (Version 1.0)

---

1. Overview

Project Name

Formal Verification MCP (Model Context Protocol) Server

Goal

Develop a domain-specific MCP server that enables Large Language Models (LLMs) such as ChatGPT, Claude, Gemini, or internal enterprise models to interact with RTL codebases, formal verification tools, specifications, assertions, regressions, and debugging artifacts.

The MCP server should abstract all verification workflows into reusable tools so that engineers can query hardware designs using natural language instead of manually navigating multiple EDA tools.

---

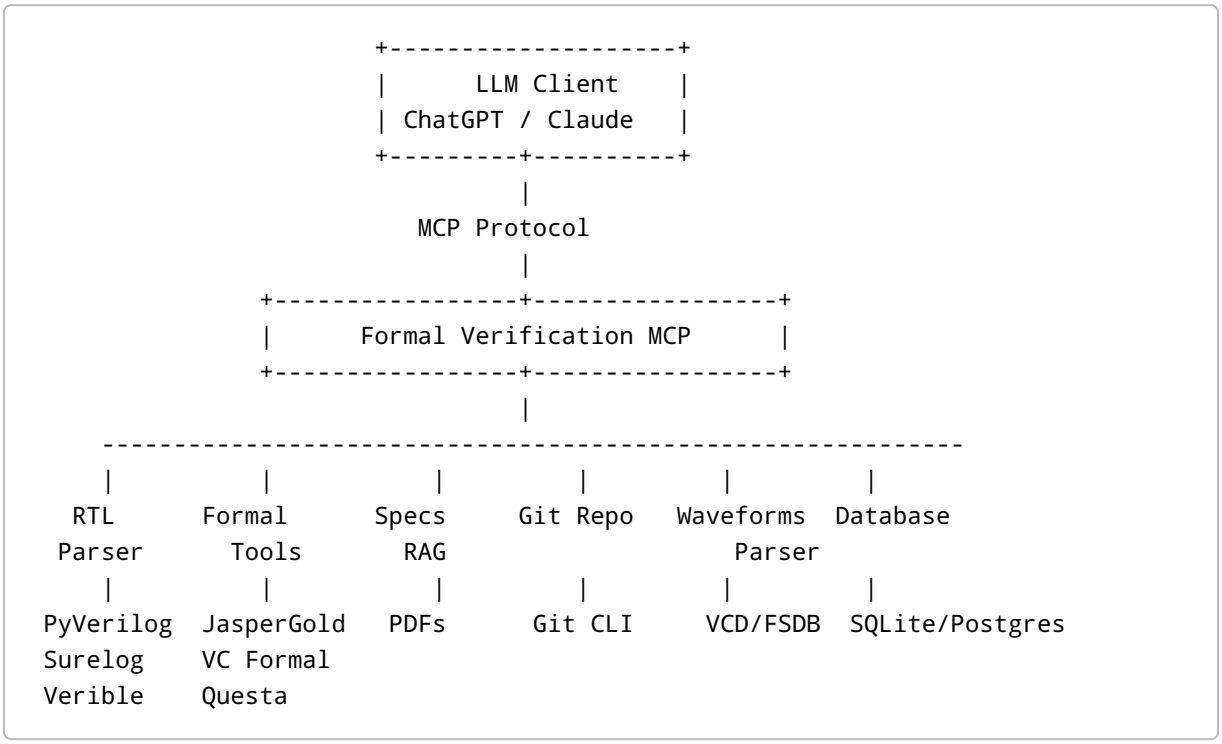
2. Objectives

The platform should enable engineers to:

- Explore RTL hierarchy
- Understand complex designs
- Generate SystemVerilog Assertions (SVAs)
- Execute formal verification
- Analyze counterexamples
- Parse waveforms
- Search protocol specifications
- Analyze coverage
- Detect vacuous properties
- Compare regressions
- Search historical bugs
- Explain assertion failures
- Recommend fixes

---

3. High-Level Architecture



---

4. Technology Stack

Backend

- Python 3.12+
- FastMCP
- FastAPI
- Pydantic
- AsyncIO

Parsing

- Surelog
- UHDM
- PyVerilog
- Verible

Formal Verification

- Cadence JasperGold

- Synopsys VC Formal
- Siemens Questa Formal

Waveforms

- PyVCD
- FSDB APIs (licensed)

Storage

- PostgreSQL
- SQLite (development)
- Redis (cache)

Vector Database

- FAISS
- Qdrant

Embeddings

- OpenAI
- Gemini
- Local embedding models

---

5. Repository Structure

```
formal-verification-mcp/

│
├── app/
│   ├── server.py
│   ├── config.py
│   ├── settings.py
│   └── logging.py
│
├── tools/
│   ├── rtl/
│   ├── formal/
│   ├── waveform/
│   ├── assertions/
│   ├── coverage/
│   ├── bugs/
│   ├── specs/
```

```
│   ├── git/
│   └── utilities/
│
├── services/
│   ├── rtl_service.py
│   ├── formal_service.py
│   ├── waveform_service.py
│   ├── rag_service.py
│   ├── embedding_service.py
│   └── regression_service.py
│
├── parsers/
│   ├── verilog_parser.py
│   ├── vcd_parser.py
│   ├── fsdb_parser.py
│   ├── report_parser.py
│   └── assertion_parser.py
│
├── models/
│   ├── rtl.py
│   ├── assertion.py
│   ├── waveform.py
│   └── regression.py
│
├── connectors/
│   ├── jasper.py
│   ├── vcformal.py
│   ├── questa.py
│   └── git.py
│
├── database/
│
├── prompts/
│
├── tests/
│
├── docs/
│
└── requirements.txt
```

---

6. MCP Tool Catalog

RTL Tools

find_module

Find module definition.

Inputs

- module_name

Returns

- hierarchy
- ports
- parameters
- instances

---

find_signal

Search signal.

Returns

- declaration
- width
- driver
- fanout
- module

---

show_hierarchy

Returns complete module hierarchy.

---

show_fsm

Extract finite state machine.

Returns

- states

- transitions
- reset state
- outputs

---

list_modules

Return all modules.

---

find_clock_domains

Identify

- clocks
- generated clocks
- asynchronous domains

---

find_reset_tree

Return reset topology.

Assertion Tools

generate_assertion

Input

Natural language requirement.

Output

SystemVerilog Assertion.

---

explain_assertion

Input

SVA property.

Output

Plain English explanation.

---

lint_assertion

Checks

- syntax
- unsupported constructs
- vacuity risks
- overlapping implications

---

optimize_assertion

Suggest improvements.

Formal Tools

run_formal

Inputs

- RTL
- assertions
- constraints

Outputs

- pass/fail
- runtime
- counterexample
- proof depth

---

generate_constraints

Automatically infer

- clocks
- resets
- assumptions
- black boxes

analyze_counterexample

Input

Counterexample trace.

Returns

- failure cycle
- violated signals
- root cause summary

---

compare_runs

Compare two formal executions.

Waveform Tools

parse_vcd

Load VCD.

---

parse_fsdb

Load FSDB.

---

explain_waveform

Generate natural-language explanation.

---

signal_history

Return signal values across cycles.

---

compare_waveforms

Compare two traces.

Specification Tools

search_spec

Search protocol documents.

Supports

- AXI
- APB
- PCIe
- USB
- DDR
- RISC-V

---

explain_protocol

Explain protocol behavior.

---

requirement_to_property

Convert requirement into SVA.

Coverage Tools

analyze_coverage

Returns

- property coverage
- activation coverage
- proof coverage

---

uncovered_properties

List uncovered assertions.

---

recommend_tests

Suggest stimulus to improve coverage.

Bug Database

search_bug

Semantic bug search.

---

similar_failures

Find matching historical failures.

---

regression_summary

Summarize overnight regressions.

Git Tools

search_commits

Find commits affecting modules.

---

blame_signal

Identify commit history.

---

recent_changes

Return latest modifications.

Utility Tools

summarize_design

Generate architecture summary.

---

explain_module

Explain functionality.

---

generate_documentation

Generate Markdown documentation.

---

dependency_graph

Return dependency graph.

---

7. Services Layer

RTLService

Responsibilities

- Parse RTL
- Cache AST
- Extract hierarchy
- Find FSMs
- Signal lookup

FormalService

Responsibilities

- Execute formal tools
- Parse reports
- Collect logs
- Aggregate results

WaveformService

Responsibilities

- Read VCD
- Read FSDB
- Timeline reconstruction
- Signal extraction

RAGService

Responsibilities

- Embed specifications
- Semantic search
- Context retrieval

RegressionService

Responsibilities

- Compare regressions
- Generate summaries
- Detect new failures

---

8. Example MCP Calls

Example 1

User

```
Show me the AXI write state machine.
```

Flow

```
LLM

↓

show_fsm()

↓

RTL Parser

↓

FSM JSON

↓

LLM Explanation
```

Example 2

User

```
Generate assertions for FIFO overflow.
```

Flow

```
LLM

↓

generate_assertion()

↓

SVA

↓
```

```
LLM Review
```

Example 3

User

```
Why did property p_fifo_order fail?
```

Flow

```
run_formal()

↓

counterexample

↓

parse_waveform()

↓

find_signal()

↓

git search

↓

root cause
```

---

9. Data Models

Module

```
Module
- name
```

```
- ports
- parameters
- signals
- instances
- source_file
```

Signal

```
Signal
- name
- width
- type
- drivers
- loads
```

Assertion

```
Assertion
- name
- property
- module
- status
- coverage
```

CounterExample

```
CounterExample
- property
- cycle
- signals
- explanation
```

---

10. Future AI Agents

RTL Agent

Explains RTL.

Assertion Agent

Writes SVAs.

Debug Agent

Explains failures.

Coverage Agent

Improves proof coverage.

Regression Agent

Summarizes nightly regressions.

Spec Agent

Answers protocol questions.

Documentation Agent

Generates design documentation.

---

11. Security

- Workspace sandboxing
- Read-only source access by default

- Role-based permissions
- Audit logs for tool invocations
- Secure handling of licensed EDA tool paths and credentials
- Configurable access to proprietary specifications and IP

---

12. Deliverables

Phase 1 (MVP)

- FastMCP server
- RTL parser
- Module explorer
- Signal explorer
- SVA generator
- Specification RAG
- Formal execution wrapper
- Counterexample parser

Phase 2

- Waveform reasoning
- Coverage analysis
- Regression comparison
- Git integration
- Bug similarity search

Phase 3

- Multi-agent orchestration
- Automatic root cause analysis
- Assertion optimization
- Requirement-to-verification planning
- Dashboard and reporting
- IDE integration (VS Code)
- CI/CD integration for nightly formal verification

---

13. Success Metrics

- Reduce assertion authoring time by >50%
- Reduce debug time for formal failures by >40%

- Improve assertion reuse across projects
- Provide natural-language access to RTL and verification assets
- Enable semantic search across specifications, bugs, and regressions
- Integrate seamlessly with enterprise formal verification workflows

---

14. Stretch Goals

- Automatic verification plan generation from specifications
- AI-assisted constraint synthesis
- Formal proof explanation with timing diagrams
- Requirement traceability (Spec → Assertion → Proof → Coverage)
- Multi-design comparison and IP reuse analysis
- Knowledge graph linking RTL, assertions, bugs, commits, and specifications
- Support for cloud-based formal verification farms